

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ
СИСТЕМ**

Лабораторна робота №5

Автоматизація тестування клієнту та серверу.

Виконав: ст. гр. КІ-404
Гірняк О.Р.
Прийняв:
Федак П.Р.

Львів 2024

Мета роботи: Ознайомитись з написанням автоматизованих тестів з використанням власноруч створеного або існуючого фреймворку для тестування.

ЗАВДАННЯ

Task 5. Implement automated tests:

1. Implement or use existing test framework;
2. Create a set of automated tests;
3. Test report should contain number of all tests, passed tests, failed tests, coverage;
4. Coverage must be more than 80%
5. Required steps

Індивідуальне завдання:

Згідно списку групи я маю 2 варіант, так як я маю такий порядковий номер у списку.

Student number	Game	config format
2	rock paper scissors	JSON

Виконання роботи

1. Для тестування Arduino обрав бібліотеку ArduinoUnit. Її потрібно встановити в Arduino IDE.

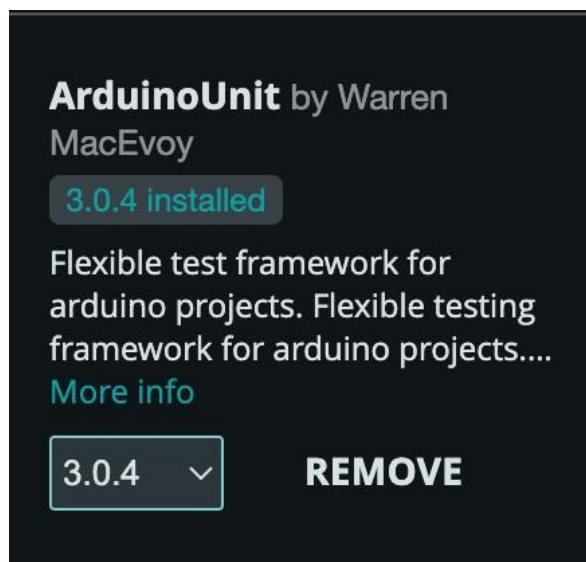


Рис.1. Обрана бібліотека для тестування Arduino програми

2. Для тестування клієнтського коду (TypeScript + React) використовую Vitest та дві додаткові бібліотеки (jest-dom та react) для тестування TSX компонентів.

3. Написав тести для серверної частини. Протестував всі складові системи з усіма їхніми публічними методами. Для створення тестів використовується спеціальна конструкція `test(test_name){ test_body }`, яка дуже схожа на звичайне оголошення функції.

```
test(GameManager_setGameMode) {
  GameManager* gameManager = GameManager::getInstance();
  gameManager->setGameMode("hvc");
  GameState state = gameManager->getGameState();
  assertEquals(state.gameMode, "hvc");
  assertEquals(state.player1Score, 0);
  assertEquals(state.player2Score, 0);
}

test(GameManager_processMove_hvh) {
  GameManager* gameManager = GameManager::getInstance();
  gameManager->setGameMode("hvh");
  gameManager->processMove("rock", true);
  gameManager->processMove("scissors", false);

  GameState state = gameManager->getGameState();
  assertEquals(state.player1Score, 1);
  assertEquals(state.player2Score, 0);
  assertEquals(state.lastMove1, "");
  assertEquals(state.lastMove2, "");
}
```

Рис.2. Приклад тестів серверної частини

4. Під'єднав Arduino та запустив тестування. Бібліотека ArduinoUnit запускає тести на самій платі та виводить у серійний порт результати тестування.

Жодних додаткових дій окрім написання тестів не потрібно.

```
Test GameManager_loadGame passed
Test GameManager_processMove_cvcs passed
Test GameManager_processMove_hvh passed
Test GameManager_resetScores passed
Test GameManager_saveGame passed
Test GameManager_setGameMode passed
Test GameManager_singleton passed
Test GameManager_updateScore passed
Test gameState_initialization passed
Test randomStrategy_makeMove passed
Test smartStrategy_adaptiveMove passed
Test smartStrategy_initialMove passed
Test summary: 12 passed, 0 failed, 0 skipped, out of 12 test(s).
```

Рис.3. Підсумок тестування серверної частини.

5. Написав тести для клієнтської частини гри. Оскільки ми пишемо unit-тести, то протестувати в нашій програмі можливо все окрім основного компоненту, тому що свою логіку він інкапсулює, а також він є простою вхідною точкою програми, яка об'єднує всі компоненти для реалізації логіки гри. Написання тестів на Vitest відрізняється від написання тестів на Arduino. При використанні Vitest тести описуються спеціальними функціями, які названі так, що їх можна порівнювати зі звичайною людською мовою, цьому також допомагає сама структура виклику функцій (наприклад `it("renders the Select with provided options", ... expect(screen.getByText(option.label)).toBeInTheDocument();`). Така структура допомагає краще розуміти тест інтуїтивно. На рис. 5 продемонстрований приклад.

Приклад тестів для коду клієнтської частини:

```
describe("Select component", () => {
  it("renders the Select with provided options", () => {
    const options = [
      { value: "option1", label: "Option 1" },
      { value: "option2", label: "Option 2" },
      { value: "option3", label: "Option 3" },
    ];
    render(<Select options={options} onChange={() => {}} value="option1" />);

    options.forEach((option) => {
      expect(screen.getByText(option.label)).toBeInTheDocument();
    });
  });
});
```

```

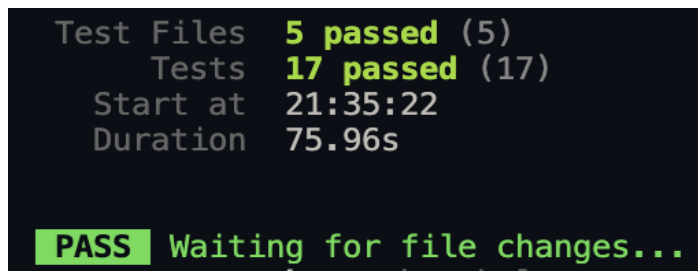
it("calls the onChange callback with the selected value", () => {
  const options = [
    { value: "option1", label: "Option 1" },
    { value: "option2", label: "Option 2" },
    { value: "option3", label: "Option 3" },
  ];
  const onChange = vi.fn();
  render(<Select options={options} onChange={onChange} value="option1" />);

  fireEvent.change(screen.getByRole("combobox"), {
    target: { value: "option2" },
  });

  expect(onChange).toHaveBeenCalled("option2");
});
});

```

1. Виконав тести командою `prx vitest`. В звіті про тестування можемо бачити кількість файлів з тестами та кількість самих тестів, а також скільки з них були пройдені успішно. У нашому випадку всі тести були пройдені успішно.



```

Test Files  5 passed (5)
Tests      17 passed (17)
Start at   21:35:22
Duration   75.96s

PASS Waiting for file changes...

```

Рис.4. Результати тестування клієнтської частини

2. Аналіз покриття тестами коду. Для аналізу покриття тестами клієнтської частини встановив додатковий пакет `Vitest Coverage`. Для виведення інформації

про покриття разом з тестами, використовується команда `prx vitest run --coverage`.

Результат:

% Coverage report from v8					
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
ui/libs/components	100	100	100	100	
components.ts	100	100	100	100	
ui/libs/components/button	91.66	50	100	91.66	
button.tsx	91.66	50	100	91.66	40,42
ui/libs/components/card	100	100	100	100	
card.tsx	100	100	100	100	
ui/libs/components/select	100	75	100	100	
select.tsx	100	75	100	100	41
ui/libs/components/serial-context-provider	30.33	66.66	20	30.33	
serial-context-provider.tsx	30.33	66.66	20	30.33	47-66, 69-84, 87-92, 97-120
ui/libs/hooks	64.7	100	50	64.7	
hooks.ts	100	100	100	100	
use-game-state.hook.ts	25	100	0	25	11-18
use-serial-communication.hook.ts	100	100	100	100	
ui/libs/types	0	0	0	0	
types.ts	0	0	0	0	

Рис.5. Результати тестування клієнтської частини

Для аналізу покриття тестами серверної частини, в бібліотеки ArduinoUnit немає спеціальних інструментів тому покриття будемо визначати вручну. Оскільки тестами покриті всі методи розроблених класів, окрім основних методів Arduino **setup** і **loop** можемо зробити висновок що покрито більше ніж 90% коду.

Отже, весь код покритий тестами більше ніж на 80%, що відповідає вимогам до даної лабораторної роботи.

Висновок:

У процесі виконання лабораторної роботи я набув практичного досвіду у тестуванні Arduino-коду з використанням бібліотеки ArduinoUnit та тестуванні веб-додатка на TypeScript і React за допомогою Vitest. Я ознайомився з принципами юніт-тестування для апаратних проектів та веб-розробки, а також навчився організовувати та проводити тести для перевірки коректності функціональності на різних платформах

